

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

I Sneka SM (19232108) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Sneka
Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	5	3	3	2	16
Q2	2	4	4	3	2	15
Q3	3	4	3	4	1.5	15.5
Q4	4	3	2	3	1.5	13.5
Q5	2	4	4	2	2	14

74

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements ⁴	Good understanding with minor gaps ³	Basic understanding; some requirements unclear ²	Poor understanding or misinterpretation of the problem ¹
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems ⁶	Applies relevant concepts correctly with minor errors ⁵	Applies basic concepts with limited accuracy ⁴	Fails to apply appropriate concepts ¹⁻²
Problem-Solving Approach	20	Logical, well-structured, and efficient approach ⁴	Mostly logical approach with minor gaps ³	Basic approach with some inconsistencies ²	Illogical or unclear approach ¹
Accuracy of Solution	20	Completely correct solution with proper justification ⁴	Mostly correct with minor errors ³	Partially correct solution ²	Incorrect or incomplete solution ¹
Clarity	10	Clear, well-organized, and properly explained solution ²	Understandable with minor clarity issues ^{1.5}	Limited clarity and explanation ¹	Poorly presented and difficult to understand ⁰

QUESTION 1

Computer performance depends on the coordinated operation of the CPU, memory, and I/O subsystem. The CPU executes instructions, memory stores instructions and data, and I/O devices transfer information between the computer and external devices. When the amount of data transfer increases, delays may occur because of memory latency, I/O waiting time, and bus contention.

Smart City Surveillance System

A smart city surveillance system receives continuous video data from many cameras. This data must move through I/O devices → memory → CPU, where it is analyzed for threats. During peak hours, the huge amount of incoming data can cause memory delays, I/O waiting, and bus contention, resulting in lag.

1. Interaction between CPU, Memory and I/O

1. I/O devices: Receive video data from multiple cameras.
2. Memory: Temporarily stores the incoming video frames and required program data.
3. CPU: Fetches instructions and data from memory and performs threat detection.
4. When many cameras send data simultaneously, I/O traffic increases.
5. Frequent memory accesses increase memory stall time.
6. If data arrives faster than the CPU can process it, frames accumulate in buffers, increasing response time.
7. Cache memory can reduce memory-access delays, while DMA can transfer large blocks of data between I/O and memory with less CPU involvement.

Basic flow:

Cameras → I/O → Memory → CPU → Threat Detection

2. Single-Bus Structure

In a single-bus structure, CPU, memory and I/O devices share one common communication path.

Advantages:

- Simple architecture
- Less hardware
- Lower cost

Problem: All components compete for the same bus. During peak hours, many transfers happen simultaneously, causing bus contention and making the bus a bottleneck.

Result:

Heavy traffic → Bus contention → Waiting → Delayed threat detection

3. Multi-Bus Structure

A multi-bus structure provides multiple communication paths between system components.

It allows greater parallel data transfer and reduces interference between CPU, memory and I/O operations.

Advantages:

- Less bus contention
- Higher bandwidth
- More parallel transfers
- Better overall throughput

Its main disadvantage is greater hardware complexity and cost.

Therefore, for a surveillance system with multiple cameras, a multi-bus structure is more suitable for handling heavy traffic.

4. Improving the Instruction Execution Cycle

The CPU generally performs:

Fetch → Decode → Execute → Store

Performance can be improved using:

1. Instruction Pipelining – Overlaps different instruction stages, increasing throughput.
2. Cache Memory – Keeps frequently used instructions/data closer to the CPU.
3. Branch Prediction – Reduces delays caused by conditional branches.
4. Out-of-Order Execution – Executes independent instructions while another instruction is waiting.
5. Superscalar Execution – Allows multiple instructions to execute during one clock cycle.
6. Multicore Processing – Allows different workloads to execute simultaneously.
7. DMA – Transfers large data blocks without continuous CPU involvement.

5. Overall Effect

Single bus + heavy video traffic

- Bus contention
- CPU/memory waiting
- Lower throughput
- Delayed threat detection

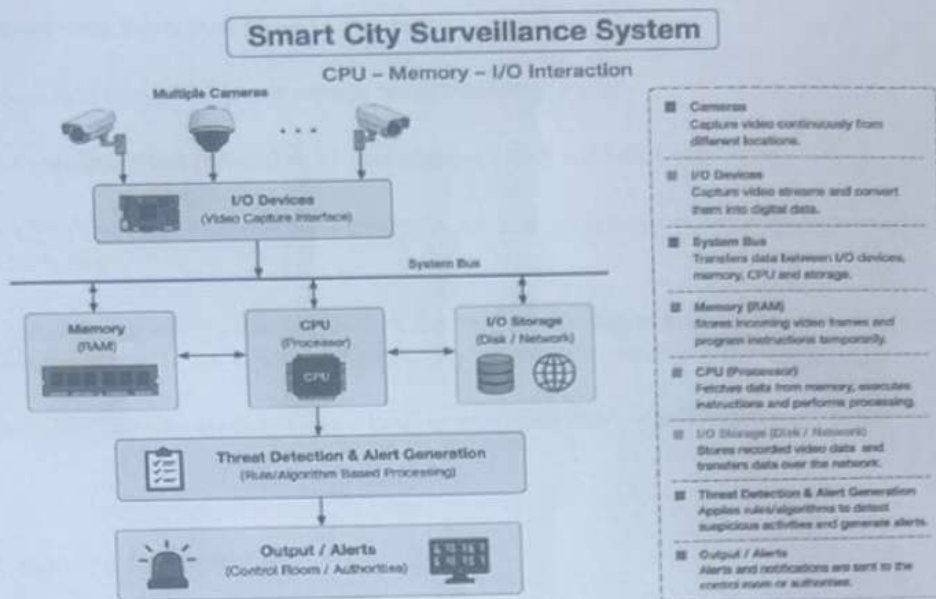
Whereas:

Multi-bus + cache + DMA + pipelining + multicore processing

- Less contention
- Faster data movement
- Better CPU utilization
- Faster threat detection

Conclusion

The lag occurs because CPU, memory and I/O must handle a very large amount of data simultaneously. A single-bus system can become a bottleneck because all components share the same communication path. A multi-bus system provides better parallelism and bandwidth. Along with pipelining, cache memory, branch prediction, out-of-order execution, superscalar processing, multicore CPUs and DMA, system throughput and real-time threat detection can be significantly improved.



QUESTION 2

The fetch-decode-execute cycle is the fundamental process through which a processor executes instructions. CPU performance depends not only on clock frequency but also on the number of clock cycles required to execute each instruction. This is represented by Cycles Per Instruction (CPI).

1. Instruction Fetch

1. The **Program Counter (PC)** stores the location of the next instruction.
2. This address is provided to the **cache or memory unit**.
3. The processor retrieves the required instruction.
4. The instruction is transferred to the **Instruction Register (IR)**.
5. The PC advances to the address of the following instruction.

2. Instruction Decode

1. The **Control Unit** receives the instruction from the IR.
2. It identifies the **opcode and type of operation**.
3. The processor identifies the required **source registers and data operands**.
4. Control signals are produced for the required operation.
5. The instruction is directed toward the appropriate **functional unit**.

3. Instruction Execution

1. The selected execution unit performs the requested operation.
2. The **ALU** performs arithmetic and logical calculations.
3. Load and store instructions communicate with the memory hierarchy.
4. A conditional branch can change the **next instruction address**.
5. The generated result is written to a register or memory location.

4. Relationship Between CPI and CPU Time

The execution time of the processor can be expressed as:

$$\text{CPU Execution Time} = \text{Number of Instructions} \times \text{CPI} \times \text{Clock Cycle Time}$$

Here, CPI (Cycles Per Instruction) represents the average number of clock cycles needed to complete one instruction.

In a stock trading application, millions of instructions are processed continuously. Therefore, even a small increase in CPI can result in a significant increase in processing time.

Higher CPI $\rightarrow$ More clock cycles $\rightarrow$ Longer execution time $\rightarrow$ Delayed transactions

5. Reasons for Increased CPI

1. **Cache Misses**
Required data is not present in the cache, so the processor must access slower memory.
2. **Incorrect Branch Prediction**
If the processor predicts the wrong branch, already fetched instructions may have to be removed.
3. **Instruction Dependencies**
An instruction may have to wait for the output of an earlier instruction.
4. **Pipeline Delays**
Stalls can leave some pipeline stages temporarily unused.
5. **Memory Latency**
Slow memory responses cause the CPU to wait for required data.
6. **Resource Conflicts**
Multiple instructions may require the same execution resource at the same time.

6. Architectural Techniques to Improve Performance

1. Improved Cache Organization

Using appropriate L1, L2 and L3 caches allows frequently accessed trading instructions and data to remain closer to the processor.

Result: Reduced memory waiting time and lower CPI.

2. Advanced Branch Prediction

The processor can predict the likely outcome of conditional instructions.

Correct prediction $\rightarrow$ Continuous instruction flow $\rightarrow$ Fewer pipeline interruptions

3. Pipeline Optimization

The instruction process can be divided into several stages so that multiple instructions are handled simultaneously.

Result: Better instruction throughput.

4. Out-of-Order Processing

When one instruction is waiting for data, the processor can execute another independent instruction.

Result: Execution units remain busy instead of waiting.

5. Data Prefetching

The processor anticipates which data may be required next and loads it into cache in advance.

Result: Less time waiting for memory.

6. Multiple-Issue / Superscalar Processing

The processor can issue more than one independent instruction during a clock cycle.

Result: More instructions completed in less time.

7. Instruction Scheduling

Instructions can be arranged efficiently to minimize dependencies and avoid unnecessary pipeline stalls.

Result: Better utilization of CPU resources.

7. Performance Improvement in the Trading System

Without Optimization

Frequent Memory Access



Cache Misses / Memory Waiting



Pipeline Stalls

Frequent Branches



Branch Mispredictions



Pipeline Flushes

Both lead to:

Higher CPI → Increased CPU Time → Transaction Delays

With Architectural Enhancements

Better Cache + Branch Prediction + Pipelining + Prefetching + Out-of-Order Execution + Superscalar Processing



Fewer stalls and better resource utilization



Reduced CPI

↓

Higher CPU throughput

↓

Faster transaction processing

Conclusion

In a high-speed stock trading application, the instruction execution cycle directly affects how quickly transactions can be processed. Frequent memory operations and branch decisions can cause cache misses, pipeline stalls, and wasted cycles, resulting in a high CPI.

Using better cache organization, branch prediction, optimized pipelining, out-of-order execution, prefetching, superscalar processing, and efficient instruction scheduling can reduce unnecessary CPU cycles. This improves processor throughput and allows the trading system to respond faster even under heavy workloads.

QUESTION 3

The 8085 microprocessor provides different addressing modes to specify the location or source of operands. The choice of addressing mode determines how the processor obtains data and affects memory access, instruction length, and execution

1. Immediate Addressing

In this method, the actual data is provided along with the instruction. The processor does not need to search for the value elsewhere.

Example:

MVI A, 35H

Here, 35H is the data that is directly loaded into the accumulator.

Key point: Data is available directly in the instruction.

2. Register Addressing

In register addressing, the required operand is already present in a processor register.

Example:

MOV A, C

The contents of register C are copied into register A.

Key point: Register access is fast because the data is already inside the CPU.

3. Direct Addressing

In direct addressing, the memory address containing the required data is explicitly mentioned in the instruction.

Example:
LDA 3050H

The contents of memory location 3050H are transferred to the accumulator.

Key point: The programmer specifies the exact memory location.

4. Register Indirect Addressing

Here, the address of the required memory location is held inside a **register pair**.

Example:
MOV A, M

The **HL register pair** contains the address of the memory location. The processor reads the data from that location and places it in the accumulator.

Key point: The register pair acts as a pointer to memory.

5. Implied Addressing

In implied addressing, the operand is **not explicitly mentioned** because the instruction already specifies which operand is involved.

Example:
CMA

This instruction complements the contents of the **accumulator**.

Key point: The operand is automatically understood by the processor.

Influence of Addressing Modes on Data Access

1. Addressing modes specify **where the operand is located**.
2. **Immediate addressing** obtains data directly from the instruction.
3. **Register addressing** provides quick access to data stored in CPU registers.
4. **Direct addressing** is useful when a specific memory location is known.
5. **Register indirect addressing** allows memory to be accessed using an address stored in a register pair.
6. The selected addressing mode can affect instruction length and the number of memory accesses.

Causes of Incorrect Data Processing

Incorrect results in an 8085 program may occur due to:

1. **Wrong memory address** – The processor may read data from an unintended location.
2. **Uninitialized register pair** – Register indirect addressing may access an incorrect memory location.
3. **Confusing data with an address** – Immediate data and memory addresses may be incorrectly interpreted.
4. **Wrong register selection** – The instruction may operate on an unintended register.

5. **Unwanted register modification** – An important intermediate value may be overwritten.
6. **Incorrect instruction order** – Operations may occur in the wrong sequence.
7. **Incorrect jump condition** – The program may branch to the wrong section.
8. **Failure to preserve registers** – Important values may be lost before they are needed.

8085 Instruction Categories

The 8085 instruction set can be grouped according to their purpose.

1. Data Transfer Instructions

Used to move data between registers, memory and the accumulator.

Examples: MOV, MVI, LDA, STA

2. Arithmetic Instructions

Used to perform mathematical operations.

Examples: ADD, SUB, INR, DCR

3. Logical Instructions

Used for logical and comparison operations.

Examples: ANA, ORA, XRA, CMP

4. Branching Instructions

Used to change the normal sequence of program execution.

Examples: JMP, JZ, JNZ, JC

5. Stack Instructions

Used to save and restore data using the stack.

Examples: PUSH, POP

6. Control Instructions

Used to control processor operation.

Examples: NOP, HLT

Simple Comparison of Addressing Modes

Addressing Mode	Operand Location	Example
Immediate	Inside instruction	MVI A, 35H
Register	CPU register	MOV A, C
Direct	Specified memory location	LDA 3050H
Register Indirect	Memory address stored in register pair	MOV A, M
Implied	Automatically understood	CMA

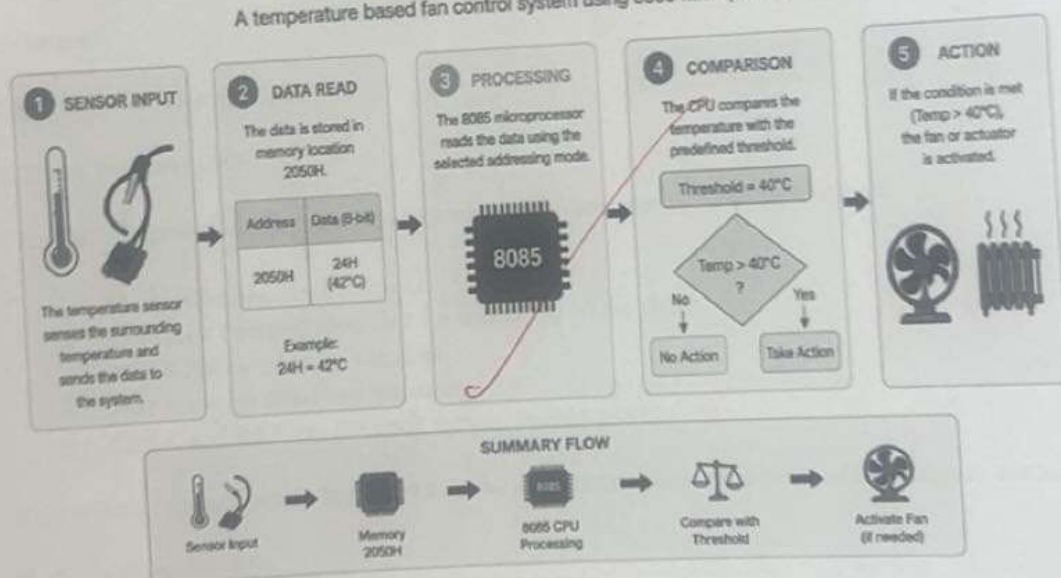
Conclusion

The addressing mode determines how the 8085 obtains its operand. Immediate and register addressing provide simple and fast data access, while direct and register indirect addressing are useful for accessing memory. Implied addressing is used when the operand is automatically understood.

For reliable 8085 programming, it is important to choose the correct addressing mode, initialize registers properly, use accurate memory addresses, and select suitable instructions. This helps avoid incorrect data processing and improves program reliability.

How the System Works

A temperature based fan control system using 8085 microprocessor



QUESTION 4

The 8086 microprocessor uses segmented memory to address up to 1 MB of physical memory. Since the processor has 16-bit registers and a 20-bit address bus, segmentation is used to generate the required physical address.

1. Segment Registers

The 8086 has four main segment registers:

1. CS – Code Segment
2. DS – Data Segment
3. SS – Stack Segment
4. ES – Extra Segment

2. Code Segment (CS)

The Code Segment contains the program instructions.

1. CS stores the base address of the code segment.
2. **IP (Instruction Pointer)** contains the offset of the next instruction.
3. The processor uses the **CS:IP combination** to fetch the next instruction.

Example:

CS : IP → Instruction to be fetched

3. Data Segment (DS)

The Data Segment contains variables and other program data.

1. DS normally identifies the data segment.
2. An **offset** identifies the location of a particular data item.
3. Proper initialization of DS is required to access the correct data.

Example:

DS : Offset → Required data

4. Stack Segment (SS)

The Stack Segment is used for temporary storage and procedure-related information.

1. SS identifies the stack segment.
2. **SP (Stack Pointer)** identifies the current top of the stack.
3. **PUSH** places data onto the stack.
4. **POP** retrieves data from the stack.
5. **CALL** and **RET** use the stack during procedure execution.

Proper stack management is important because incorrect stack operations can affect program execution.

5. Extra Segment (ES)

The Extra Segment provides an additional area for storing data.

It is particularly useful during **string operations** and gives the processor another segment for data access.

6. Physical Address Calculation

The 8086 generates the physical address using:

Physical Address = Segment Address × 10H + Offset

Example:

Given:

Segment = 2000H

Offset = 0100H

Therefore:

Physical Address = 2000H × 10H + 0100H

= 20000H + 0100H

= 20100H

So, the physical address is 20100H.

7. Problems Caused by Improper Segment Handling

Incorrect segment management can cause several problems:

1. An incorrect CS value may cause instructions to be fetched from the wrong memory location.
2. An incorrect DS value may cause the program to access incorrect variables.
3. An incorrect SS value can damage stack operations.
4. An incorrect SP value can result in invalid stack access.
5. Too many PUSH operations can consume available stack space.
6. Unbalanced PUSH and POP operations can disturb program execution.
7. Incorrect segment assumptions may cause unintended operands to be accessed.
8. Poor stack management can overwrite other memory areas and cause unpredictable results.

8. Instruction Execution in 8086

The general instruction execution process is:

Fetch → Decode → Operand Access → Execute → Store Result

The 8086 divides these responsibilities between two major units:

Bus Interface Unit (BIU)

- Handles instruction fetching.
- Performs bus-related operations.
- Works with segment registers to generate addresses.

Execution Unit (EU)

- Decodes instructions.
- Performs the required operations.
- Executes instructions using the available execution resources.

9. Addressing Modes of 8086

Important addressing modes include:

1. Immediate Addressing
2. Register Addressing
3. Direct Addressing
4. Register Indirect Addressing
5. Based Addressing
6. Indexed Addressing
7. Based-Indexed Addressing

Selecting the appropriate addressing mode ensures that the processor obtains the **correct operand** from the intended location.

10. Simple Memory Organization

You can remember the four segments like this:

8086 MEMORY

- CS → Program Instructions
- DS → Variables and Data
- SS → Stack / Temporary Data
- ES → Additional Data / String Operations

Conclusion

The 8086 uses segmentation to organize its 1 MB memory into different logical areas such as **Code, Data, Stack, and Extra** segments. The physical address is generated using the **segment address and offset**.

Correct initialization of CS, DS, SS and ES, proper management of SP, correct instruction execution, and appropriate addressing-mode selection are essential for **accurate memory access and reliable program execution**.

QUESTION 5

Digital computers internally process information using binary numbers, whereas hexadecimal provides a compact and readable representation of binary values. Conversion between hexadecimal and binary is therefore important in low-level programming, debugging, memory inspection, and digital communication.

1. Relationship Between Hexadecimal and Binary

Each hexadecimal digit corresponds exactly to four binary bits.

For example:

- A → 1010
- 5 → 0101
- F → 1111

Therefore:

A5H → 1010 0101₂

This direct relationship makes conversion between the two number systems simple.

DATA PROCESSING FLOW

From Hexadecimal Input to Binary Output using CPU

